

Negotiate, Then Solve

Solver-Grounded Negotiation for Conflict-Aware University Timetabling

Domain: institute management (academic timetabling)

Target users: academic office, Dean / HoD, faculty, lab in-charges and exam cell, students

Capabilities: fine-tuning, RAG, reasoning, agentic abilities, tool calling

Team Farali Chevdo

Achyutananda Sahoo 2025202028 Rakesh C 2026901015

Satyajit Priyadarshi 2025201055 Shubhadeep Mandal 2025201056

Abstract. University timetables change constantly after publication, and change requests arrive as unstructured text. LLMs can read such requests but do not provide a formal guarantee of hard-constraint satisfaction; constraint solvers guarantee validity but cannot read requests, explain infeasibility or negotiate. We study *solver-grounded negotiation*: when requests are infeasible, minimal unsatisfiable subsets (MUS) locate the conflict, minimal correction sets (MCS) define the candidate concessions, and OR-Tools CP-SAT re-solves each relaxation into a concrete timetable. An LLM negotiator may offer *only* these solver-verified options, ranked by a published priority policy and a concession ledger, and explains each in terms traceable to solver evidence. A human coordinator approves every change. We compare against the same agent negotiating with LLM-invented alternatives, both unverified and filtered by the same solver, on planted-conflict scenarios with stakeholder simulators that hide their true flexibility.

1 Problem and Motivation

A university timetable is built once per semester but changes weekly. Faculty report unavailability, labs close for maintenance, guest lecturers arrive with fixed windows and students report elective clashes. These requests arrive as unstructured text through the portal, email and messaging apps, and the academic office resolves them by hand; one change often silently breaks another constraint. When two requests cannot both be satisfied, there is no systematic way to decide *who should concede*, so decisions look arbitrary to those affected. Existing tools either check a finished timetable or rebuild it from scratch; none reads the request, explains why it cannot be met, or negotiates who gives way.

Why it matters. A single double-booked room disrupts whole sections for weeks, most requests have a safe resolution that could be found in seconds, and stakeholders accept compromises more readily when they can see why one was chosen and that concessions rotate.

1.1 Domain and Target Users

Domain: institute management systems, specifically academic course timetabling, scoped to one department (≈ 30 faculty, 60 sections, 20 rooms, 5-day $\times$ 8-slot week). **Target users** are five stakeholder groups with different needs and authority; their authority determines who may relax which tier of constraint (Section 4).

Stakeholder	What they need from the system	Authority
Coordinator (academic office)	Verified changes and conflict summaries	Approves and publishes
Dean / HoD	Short briefs for policy exceptions and deadlocks	Dean: T1 policy; HoD: dept.
Faculty (incl. visiting)	Plain-language requests; explained outcomes	Own classes only
Lab in-charge, exam cell	Report outages; protect exam blocks	Rooms; T2 calendar blocks
Students / class reps	Report clashes and get a tracked response	Raise issues only

1.2 What Is Missing

Prior approaches provide several of these components individually. What is missing in our setting is an *interpretation layer* that connects formal infeasibility analysis to multi-stakeholder negotiation: deciding which concessions are possible, whom to ask, and how to explain it, without ever leaving the feasible space.

Approach	Limitation for this problem
Classical timetabling solvers (CP/MIP)	Need fully formal constraints; cannot read requests, explain infeasibility or negotiate
LLM scheduling directly	Reads plain language but may violate hard constraints; no formal feasibility guarantee
LLM + solver (verifier)	Solver checks LLM output; a filter drops infeasible ideas, but candidates are still whatever the LLM imagines, so valid concessions can be missed
LLM negotiation agents	Free-form offers with no model of what is feasible
MUS/MCS infeasibility diagnosis	Finds minimal conflicts and repairs, but serves a single modeller; no multi-stakeholder negotiation

2 Research Question and Hypotheses

RQ. Can solver-derived conflict sets constrain an LLM negotiator so that it resolves infeasible timetable requests with *fewer negotiation rounds* and *lower disruption* than LLM-invented options, even when those options are filtered by the same solver before they are offered?

By construction, our system never offers an infeasible option; this is a design property, not a hypothesis. The open question is whether that guarantee is worth its price: restricting options to MCS relaxations may miss creative alternatives (a third-party swap, an unusual slot) that a free-form LLM could find and a stakeholder would accept.

- **H1 (cost of invented candidates).** Unverified LLM options (B3) are solver-rejected often enough to cost extra rounds and escalations; filtered LLM options (B4) avoid that but need more regeneration calls and miss concessions that MCS enumeration covers.
- **H2 (efficiency and quality).** Against B4, ours achieves an equal or higher resolution rate with fewer rounds, less disruption and a smaller gap to an unrestricted oracle that knows every stakeholder’s hidden flexibility.
- **H3 (explanation).** Explanations grounded in the conflict set are more faithful than free-form explanations of the same options: more claims trace to a fact, fewer private reasons leak, and the text never describes an option that was not offered, without lowering acceptance. The human pilot rates clarity and acceptability.
- **H4 (fairness).** The concession ledger lowers the dispersion of concession burden across stakeholders (Gini, maximum individual burden) compared with flat weights, without lowering the resolution rate.

A scenario is *resolved* when every affected stakeholder accepts an offered alternative and the combined repair re-solves feasibly; otherwise it is *escalated* (deadlock, timeout, or a Tier 1–2 option). Every system reports resolution rate, rounds-given-resolution and escalation rate together, so no system can look efficient by escalating hard cases early. Hypotheses are directional; we report effect sizes with 95% bootstrap confidence intervals and paired tests, and negative results will be reported.

3 Proposed Approach

Figure 1 follows one request through the system: **Request** → **Parse** → **Policy check** → **Solve** → **Negotiate** → **Human approval**. Steps ①–④ turn a plain-language message into a checked constraint model; step ⑤ is the research contribution; step ⑥ keeps a human in control of publication.

- ① **Receive.** Each channel produces one **Request** record with the sender’s identity and role (experiments inject synthetic requests here).
- ② **Parse.** A router sends routine requests to a small LoRA-tuned compiler and ambiguous ones to the prompted base model; both emit the same typed constraint JSON. A deterministic authority check, not a model, decides whether the sender may make the request.
- ③ **Policy check.** Retrieved institute rules classify the request as allowed, needs approval or forbidden, with a citation.

- 4 **Solve.** CP-SAT models room, faculty, student-group and institutional constraints and repairs the published timetable with as few changes as possible. Each request constraint is guarded by an assumption literal so it can be switched off. Feasible repairs go straight to approval.
- 5 **Negotiate.** If infeasible, the rule-based ladder drives the interpretation layer (Section 4), with the LLM turning each reply into a typed tool call, until agreement or escalation.
- 6 **Approve.** The coordinator sees the diff, affected people and conflict history; only approval publishes a new, reversible version.

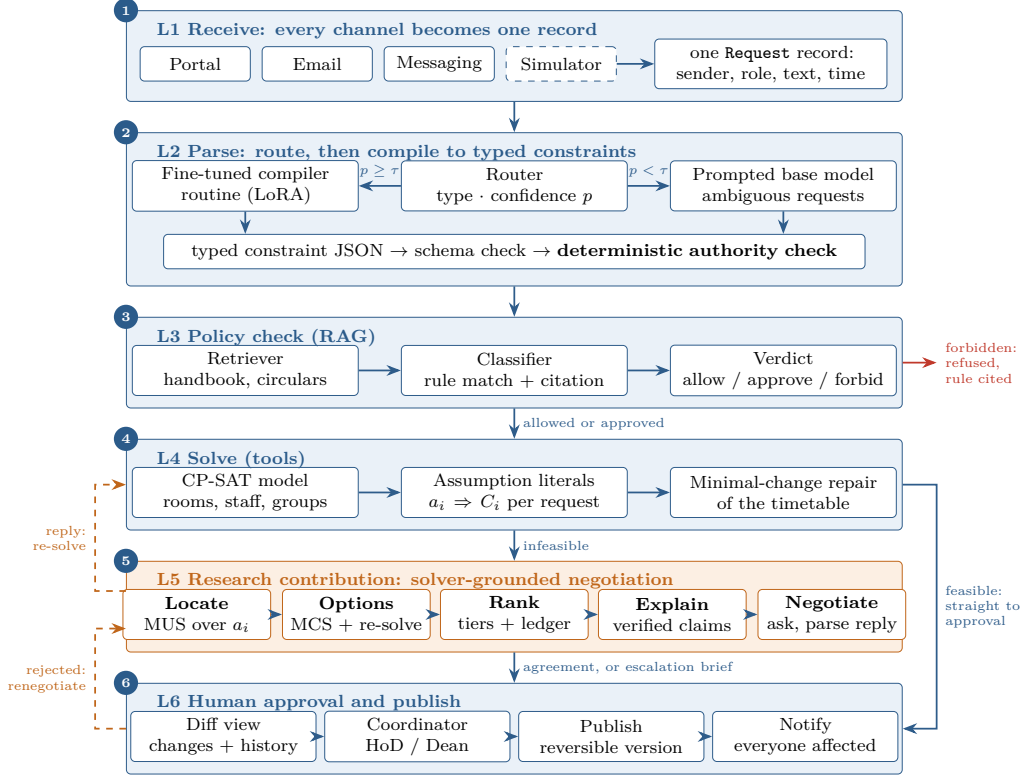


Figure 1: Architecture. Solid arrows show the main flow; dashed orange arrows are feedback loops. The orange layer (L5) is the research contribution.

4 Core Mechanism and Capabilities

4.1 Solver-Grounded Negotiation

Infeasible request → **MUS** → **MCS** → **CP-SAT re-solve** → **solver-verified alternatives** → **negotiation**

Figure 2 follows one conflict end to end. **Constraint model.** Each session is assigned exactly one time slot and one room, subject to room capacity and type, room occupancy, faculty availability, student-group clashes and institutional rules. Each request-derived constraint C_i is guarded by an assumption literal a_i , split into room and slot parts so a repair can relax one without the other. Protected T0–T2 constraints are guarded too, so they can appear in explanations, but are never relaxed. Hard constraints are solved first as a satisfaction problem, where assumption cores apply; preferences are optimised afterwards.

When the satisfaction stage is infeasible, CP-SAT reports a set of assumption literals sufficient for infeasibility, which we shrink to a **minimal unsatisfiable subset (MUS)**: *where the conflict is*. We then enumerate a few **minimal correction sets (MCS)**: minimal sets of constraints whose removal restores feasibility, i.e. *what could give way*. We enumerate them with our own MARCO-style loop over CP-SAT, restricted to negotiable literals. An MCS says which constraints yield, not what the resulting timetable is, so CP-SAT re-solves each relaxation into a concrete timetable. An option is offered only if its re-solve succeeds, so **every compromise a stakeholder sees is already verified**.

feasible. Partial concessions (“Thursday afternoon instead”) weaken a constraint rather than remove it, and trigger another re-solve. For tractability we cap enumeration at the $K=5$ cheapest candidates per round, fixed before evaluation.

Four roles are kept separate, which makes this a research question rather than “LLM + OR-Tools”:

Component	Decides
Solver (CP-SAT)	What is feasible
MCS enumeration	What concessions are possible (the candidate space)
Negotiation policy	How options rank (tier, disruption, ledger)
LLM	Language: explains, and turns each reply into one typed tool call; never adds a candidate

4.2 Priority Tiers and Negotiation Policy

Constraints fall into six published tiers. **T0–T2 are protected:** T0 is never relaxed, and any option touching T1–T2 is escalated to the named authority. **T3–T5 are negotiable** under the published priority policy, with priority scores ordering constraints within a tier.

Tier	Constraint class	Who may relax it	Example
T0	Physical impossibility	Nobody	Room double-booking; capacity
T1	Institute policy	Dean / council	Max. consecutive hours
T2	Institutional commitment	Academic office	Exam blocks; guest windows
T3	Verified unavailability	Owner + alternative	Faculty leave
T4	Operational requirement	Owner or HoD	Lab equipment; room type
T5	Preference	Owner, freely	Preferred time; fewer gaps

Resolution ladder. (i) Try a non-disruptive substitution; (ii) re-solve the $K=5$ cheapest MCS relaxations; (iii) rank feasible options by tier, disruption and increase in burden dispersion; (iv) ask an owner of a top-ranked concession; (v) turn replies into constraints and re-solve; (vi) escalate deadlock, timeout or any T1–T2 option. A persistent **concession ledger** credits each stakeholder who concedes, making them more expensive to ask next time. Weights are a published policy, not a claim of objective fairness; we test whether it spreads burden more evenly than flat weights or free negotiation. **Grounded explanations** are contrastive, and each claim must map to a constraint in the MUS, a cited rule or a solver result; a verifier rejects unsupported claims. Explanations name the *constraint*, never the private reason (Figure 2).

4.3 Control Loop, Reply Tools and Safety Boundary

The resolution ladder is the controller: it ranks options, picks whom to ask (lowest recent burden), forces T1–T2 escalation and enforces the round limit. The LLM decides only where free text must be interpreted: each stakeholder reply (“Thursday, but only after 2”, “use Lab 3 instead”, “depends on my TA”) becomes exactly one typed tool call.

Tool (LLM-selected)	Effect and guard
<code>accept(option_id)</code>	Accept an offered option; <code>option_id</code> is an enum of verified options
<code>apply_reply(constraint)</code>	Weakened or partial constraint; schema/authority-checked, then re-solved
<code>counter_propose(option)</code>	Stakeholder’s own alternative; offered onward only if CP-SAT verifies it
<code>ask_clarification(question)</code>	Reply too vague to compile; one follow-up per round
<code>decline(reason)</code>	Refusal; the ladder moves to the next-ranked option or owner
<code>escalate(reason)</code>	Out of scope, a policy exception or an injected instruction

A bad call returns a typed error; after two retries the reply goes to the coordinator. The LLM cannot create candidates, relax T0–T2 or change state.

4.4 Five Capabilities: Why Each Is Needed

Capability	Why the problem needs it	Measured by (vs. what)
Fine-tuning	Requests arrive continuously and include private staff details, so they must be compiled locally and cheaply; a prompted 2B model misreads local conventions (slot names, lab blocks)	Compilation F1: LoRA Qwen3.5-2B vs prompted 2B vs prompted Gemini Flash
RAG	Whether a request is allowed depends on this institute’s handbook and circulars, which change each semester and no model knows	Recall@k, citation precision, verdict accuracy, abstention; vs no retrieval
Reasoning	Turning a partial reply into a weakened constraint and writing a contrastive explanation are multi-step inferences over the conflict; each step is checked	Claim-level faithfulness, reply-to-constraint accuracy; A2
Agentic	Negotiation spans many rounds and people; the ledger is memory across requests; escalation is a hand-off to a human	Resolution rate, rounds, escalation precision; vs B2 (no negotiation)
Tool calling	Free-text replies must become the right typed action with valid arguments, and injected text must never trigger an action	Tool-call accuracy, argument validity, recovery, unsafe-action rate; vs A3

5 Evaluation

The primary comparison is ours against B4, which differs *only* in whether options come from re-solved MCS relaxations or from the LLM plus the same CP-SAT filter; other systems locate where any gain comes from.

5.1 Evaluation Environment

Benchmark	Construction	Size
A. Negotiation	Synthetic department instances with planted conflicts (resource contention, request vs. policy, capacity shortfall); stakeholder simulators with hidden true flexibility; solver-computed oracle outcome	60 scenarios
B. Parsing	Requests generated from known target constraints, then paraphrased, made ambiguous, unauthorised, out-of-scope or adversarial; split by template	600 (400/49/151)
C. Policy (RAG)	Questions over the handbook and circulars labelled with gold rule IDs, plus questions no rule covers (abstention)	40 + 20 queries
D. Reply actions	Stakeholder replies (accept, partial, counter-proposal, vague, refusal, injected instruction) with a known correct call and arguments	100 replies

Ground truth is known by construction. A 50-case safety set (authority violations, injections, private-reason requests) and an anonymised human pilot (5–10 faculty, informed consent, after institutional ethics clearance) complement the benchmarks.

Stakeholder simulators. Seeded hidden profiles specify acceptable options (including options outside any MCS) and whether a stakeholder volunteers an option or replies. Rules decide acceptance from the options alone; a separate LLM, blind to the negotiator prompt and solver state, only words the reply. CP-SAT runs single-threaded with a fixed seed, so paired configurations offer identical options. Four stochastic families (strict, flexible, cost-sensitive, history-sensitive) test cross-family robustness.

Judge validation. Two team members hand-label 100 explanations (claim supported or not) and 100 simulator replies; we report Cohen’s κ against the faithfulness verifier and the simulator, and report LLM-scored metrics only where $\kappa \geq 0.6$.

Models. All LLM roles use local Qwen3.5-2B Q5_K_M (GGUF) via llama.cpp: prompted for negotiation, explanation, simulation and fallback parsing; LoRA-tuned for routine compilation. Gemini Flash is the large prompted parsing reference and re-runs a subset to check model bias.

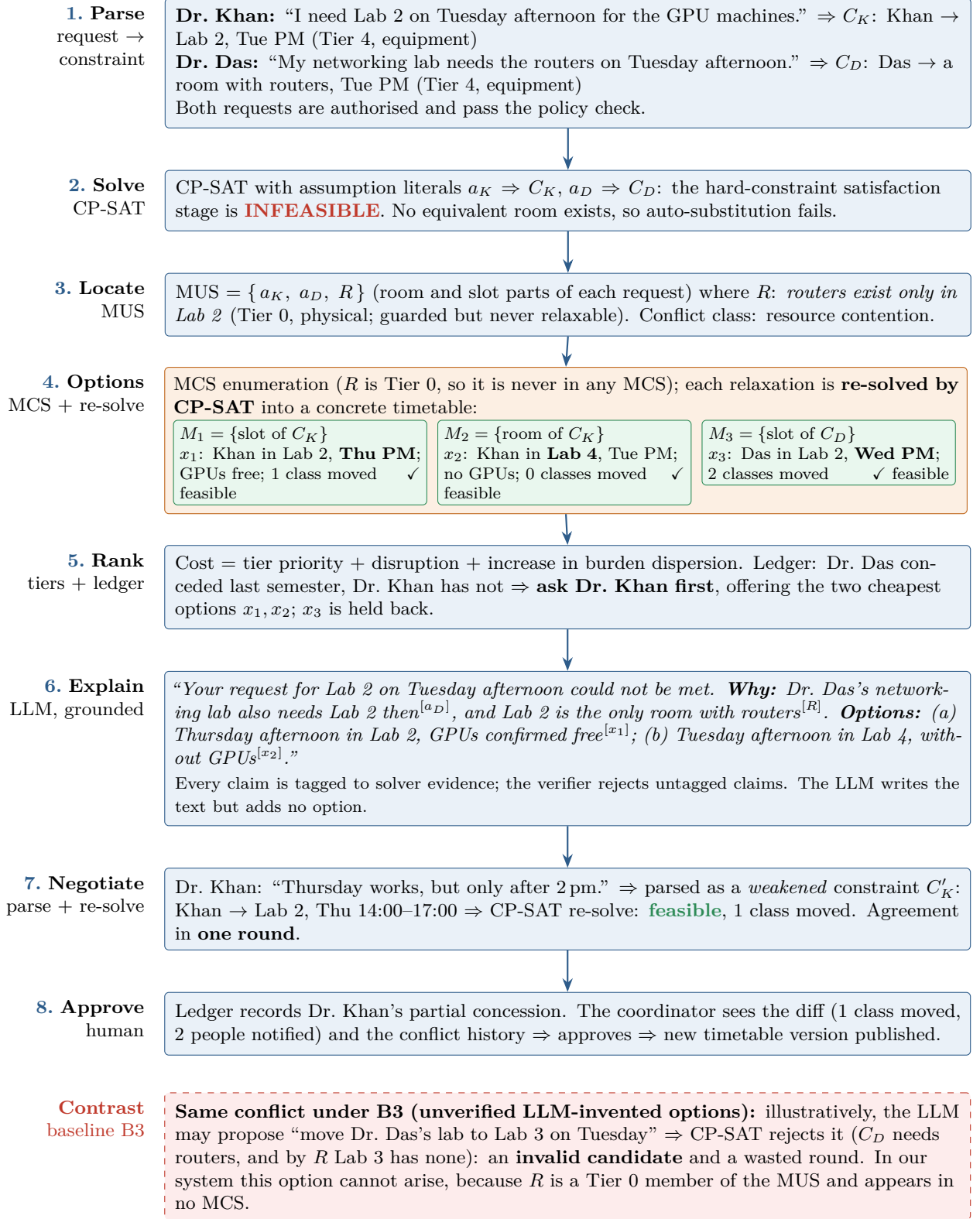


Figure 2: Worked example: one resource-contention conflict, end to end. The last row shows it under B3; B4 would discard that option and regenerate. Blue rows are deterministic or solver steps, the orange row is the MCS candidate space, and only step 6 and the parsing in steps 1 and 7 use the LLM. Superscripts mark the solver evidence behind each claim.

5.2 Baselines and Ablations

System	How it works	What it isolates
B1 LLM-only	LLM writes the timetable directly; no solver	Feasibility gap
B2 Solver only	Same model and tiers; imposes the best repair	Value of negotiation
B3 Unverified	As ours, but the LLM <i>invents</i> options, offered unchecked	Cost of invalid options
B4 Filtered	As B3, but CP-SAT filters them; LLM regenerates (≤ 3 tries)	Grounding (RQ)
Ours	Options only from re-solved MCS relaxations	—
Oracle	CP-SAT with all hidden flexibility; no MCS limit	Upper bound

Ablations: one change each, all else fixed

A1 Flat	Ours with flat stakeholder weights, no tiers or ledger	Fairness (H4)
A2 Free text	Ours with free-form instead of evidence-grounded explanations	Grounding (H3)
A3 Rule replies	Ours with a keyword/regex reply handler instead of LLM tool calls	Tool calling

5.3 Evaluation Metrics

Hyp.	vs.	Primary metrics	Supported if
H1	B3, B4	Invalid-candidate rate; regeneration calls; extra rounds/escalations	B3 needs more rounds or escalations; B4 more calls
H2	B4, B2	Resolution rate; rounds if resolved; classes moved; oracle gap	Resolution not lower; fewer rounds, less disruption, smaller gap
H3	A2	Faithfulness; leaks; text–option mismatches; acceptance	Better on the first three; acceptance not lower
H4	A1	Burden Gini; max. individual burden	Lower dispersion; resolution not lower

Per capability: compilation F1 (fine-tuning); recall@k, citation precision, verdict accuracy and abstention (RAG); claim-level faithfulness and reply-to-constraint accuracy (reasoning); resolution and escalation precision vs B2 (agentic); tool-call accuracy, argument validity, error-recovery rate vs A3 (tool calling). **Safety:** unsafe or unapproved actions, authority/tier violations, privacy leakage, injection success.

Scoring. Each scenario has one simulator family (balanced per kind). Ours, B3 and B4 run 60 scenarios $\times 3$ seeds; A1–A3 run once on a stratified 30, B1 on 20 (650 LLM runs). A final status that differs from the oracle’s is a *status error* with no oracle gap. Conclusions must hold across families; each failing run is tagged with its first failing stage.

6 Safety, Compute, Novelty and Risks

Safety. Every LLM tool call is a proposal that passes *schema validator* $\rightarrow$ *policy + authority* $\rightarrow$ *CP-SAT verify* $\rightarrow$ *human approval* before any state changes: an architectural invariant, not a prompt instruction. Messages are untrusted data, so an injection can at worst produce a proposal the checks or coordinator reject. Private reasons are never exposed; experiments use synthetic data in a sandboxed database.

Compute. LoRA training of the 2B compiler on ≈ 850 examples takes $\approx 1\text{--}2$ GPU-hours per run on a free Kaggle T4 (16 GB); we budget 3 runs, ≈ 6 GPU-hours. The 650 runs and 100 reply cases need $\approx 6,000$ short calls (≈ 2.5 GPU-hours at ≈ 1.5 s on a T4): ≈ 9 GPU-hours in total; the solver runs on a CPU. Gemini Flash (parsing reference on 151 requests; ours vs B4 on 20 scenarios to check model bias) needs ≈ 600 free-tier calls (\$0).

Expected novelty. (1) *Solver-grounded negotiation:* LLM + solver systems verify what the LLM proposes; we let MCS enumeration define the candidate space and measure the difference against generate-and-filter (H1–H2). (2) *Concession-aware policy:* published tiers set who may relax what, and a ledger carries concessions across requests and semesters, judged by burden spread (H4). (3) *Grounded, private explanations:* every claim traces to the conflict set, a rule or a solver result, checked claim by claim, with private reasons withheld (H3).

Risks $\rightarrow$ fallbacks. MCS enumeration expensive $\rightarrow$ restrict to the affected sub-problem, cap K . MCS misses acceptable options $\rightarrow$ admit verified stakeholder proposals; report the loss via oracle gap. Simulators biased or sharing a model $\rightarrow$ four families, rule-decided acceptance, Gemini Flash cross-check. Fine-tuned parser underperforms $\rightarrow$ prompted model; report the negative result. 2B model misreads replies $\rightarrow$ grammar-constrained calls, coordinator fallback. Schedule slips $\rightarrow$ cut the router, then A3, then the pilot; never Ours vs B4.